- ***4*** Additive attention
- ***5*** Saves a copy of the attention weights for later visualization
- ***6*** Batch-matrix multiplies the attention weight matrix with the node matrix to get an updated node matrix.
- ***7*** Collapses the head dimension with the feature d dimension

10.5. Double Q-learning

Now let's get to training it. Because this Gridworld environment has sparse rewards, we need to make our training process as smooth as possible, especially since we're not using curiosity-based learning.

Remember back in chapter 3 when we introduced Q-learning and a target network to stabilize training? If not, the idea was that in ordinary Q-learning we compute the target Q value with this equation:

$$Q_{new} = r_t + \gamma \times \max(Q(s_{t+1}))$$

The problem with this is that every time we update our DQN according to this equation so that its predictions get closer to this target, the $Q(s_{t+1})$ is changed, which means the next time we go to update our Q function, the target Q_{new} is going to be different even for the same state. This is problematic because as we train the DQN, its predictions are chasing a moving target, leading to very unstable training and poor performance. To stabilize training, we create a duplicate Q function called the *target function* that we can denote Q', and we use the value $Q'(s_{t+1})$ to plug into the equation and update the main Q function.

$$Q_{new} = r_t + \gamma \times \max(Q'(s_{t+1}))$$

We only train (and hence backpropagate into) the main Q function, but we copy the parameters from the main Q function to the target Q function, Q', every 100 (or some other arbitrary number of) epochs. This greatly stabilizes training because the main Q function is no longer chasing a constantly moving target, but a relatively fixed target.